GOVERNMENT OF INDIA — COPYRIGHT OFFICE

DEPOSIT OF COMPUTER SOFTWARE WORK (RULE 70, COPYRIGHT RULES, 2013)

LEXIS

Computer Software – Copyright Registration

Autonomous Multi-Agent Legal Aid System for Indian Citizens

PROJECT & COPYRIGHT IDENTIFICATION

Title of Work:	LEXIS
Class / Type of Work:	Computer Software (Literary Work under Section 2(o) of Copyright Act, 1957)
Nature of Work:	Original Application Source Code (FastAPI Backend + React Frontend)
Country of Origin:	India
Applicant / Intended Owner:	Tejas Vikas Chaudhari
Authors / Contributors (5 Total):	1. Tejas Vikas Chaudhari 2. Pratik Pandurang Devadhe 3. Manjit Ajit Bajaj 4. Vedant Jitendra Gaikwad 5. Dr. Anil Kumar Brahamane (Project Guide & Academic Mentor)
Programming Languages Found:	Python (3.11+), JavaScript (ES2022+ / React JSX), CSS3, HTML5, Shell, JSON
Backend Framework:	FastAPI (v0.115) & LangGraph (v1.2) Multi-Agent State Machine
Frontend Framework:	React 18 (v18.3) with Vite & React Router DOM (v6.27)
Databases & Storage:	MongoDB (Motor), Redis, Qdrant Vector DB, AES-256-GCM Encrypted Vault Storage

OWNERSHIP & AUTHORSHIP STATUTORY NOTE:

Ownership and authorship details must be verified against the actual agreement/assignment/NOC before submission. Under Section 17 of the Indian Copyright Act, 1957, absent an assignment deed or NOC, all 5 contributing authors remain joint first owners. If filed in the sole name of the applicant, a formal No Objection Certificate (NOC) from all co-authors is required.

CONCISE DESCRIPTION OF LEXIS (BASED ON ACTUAL REPOSITORY)

LEXIS is an autonomous multi-agent legal aid system engineered for Indian citizens. Unlike single prompt-and-response AI interfaces, LEXIS employs an asynchronous state machine built on LangGraph that decomposes citizen inquiries into discrete legal intents and coordinates specialized domain agents with concurrent superstep execution. The software provides six core functional capabilities grounded in actual statutory frameworks: (1) Rights Explainer: Delivers contextual legal explanations using Qdrant vector retrieval (BGE-M3 embeddings) with strict hallucination safeguards and citation verification; (2) Document Drafter: Generates formal legal drafts (RTI applications, consumer dispute notices, tenant notices) via clause questionnaires with native PDF and DOCX export; (3) Document Analyzer: Ingests contracts and agreements, performs PyMuPDF text extraction and Tesseract OCR, and classifies clause risks (Green/Yellow/Red) with bilingual English/Hindi explanations; (4) Scam Detection Engine: Validates suspicious communications against a 17-pattern vector seed database with fast-path matching; (5) Complaint Filing Guide: Maps citizen grievances to appropriate Indian regulatory authorities (RTI portals, NCH, District Commissions); (6) Vault & Deadline Agent: Provides AES-256-GCM encrypted document storage with proactive APScheduler deadline monitoring that triggers timely automated notice drafting at T-30, T-15, and T-3 intervals. All specialist outputs pass through a central Confirmation Agent executing bounded self-correction (max 3 retries) and cross-agent contradiction verification (Rule C1: Legal Act Mismatch) before response compilation.

STATUTORY SOURCE CODE DEPOSIT STRUCTURE (RULE 70 REQUIREMENT)

DEPOSIT PORTION	REPRESENTATIVE SOURCE FILES	PAGE RANGE	EXTENT
Part A: First 10 Pages	backend/main.py, backend/core/config/settings.py, backend/agents/orchestrator/state.py, agent.py	**Pages 2–11**	First 10 Pages (Core Backend)
Part B: Last 10 Pages	backend/services/vault/scheduler.py, frontend/src/App.jsx, frontend/src/context/AuthContext.jsx, services/api.js	**Pages 12–21**	Last 10 Pages (Services & Client)

FILE: backend/main.py LANG: Python
MODULE: FastAPI Application Entrypoint, Lifespan & Route Mounting

```
"""
╔══════════════════════════════════════════════════════════════════════╗
║                    LEXIS Backend — Main Entrypoint                   ║
║                    Module 1: Core Backend Foundation                 ║
╚══════════════════════════════════════════════════════════════════════╝

WHAT EACH MAJOR PIECE DOES
──────────────────────────
• create_app()          App factory function.  Creates and configures the
                        FastAPI instance. Using a factory (instead of a
                        bare `app = FastAPI()`) lets tests call
                        create_app() with a fresh app each time without
                        polluting global state.

• @asynccontextmanager  The "lifespan" function runs code BEFORE the app
  lifespan()            starts accepting requests (startup) and AFTER it
                        stops (shutdown).  The `yield` in the middle is
                        where the app lives.  Everything before yield =
                        startup. Everything after yield = teardown.

• app.state             FastAPI's built-in key-value store attached to the
                        app object.  We store the Mongo client, Redis
                        client, DB name, and settings here so any request
                        can access them via `request.app.state`.

• CORSMiddleware        Tells browsers which origins are allowed to make
                        requests to this API.  We read FRONTEND_ORIGIN
                        from .env — never hardcode "*" which would allow
                        ANY website to call our API.

• /api/v1 prefix        All routes are versioned.  When we need breaking
                        changes later, we can add /api/v2 alongside v1
                        without breaking existing clients.

• setup_logging()       Configures every log in the app to emit JSON so
                        the Admin Dashboard (Module 12) can parse them.

WHY THE LIFESPAN PATTERN INSTEAD OF CONNECTING PER REQUEST
──────────────────────────────────────────────────────────
If we connected to MongoDB or Redis on EVERY incoming request:
  1. Each HTTP call would pay a ~20-100 ms TCP handshake to connect.
  2. Under load (100 req/s), we'd open 100 connections/second — the DB
     server would refuse or crash.
  3. We'd need complex error-handling everywhere.

With the lifespan pattern we connect ONCE at startup, hold the connection
pool open, and every request reuses an existing connection from the pool.
This is ~1000× faster per request and uses a predictable, bounded number
of database connections regardless of traffic.

WHAT TO PUT IN YOUR .env FILE TO RUN LOCALLY
────────────────────────────────────────────
Copy .env.example to .env (cp .env.example .env) and fill in:

  SECRET_KEY          → Any long random string (run: python -c "import secrets; print(secrets.to
      ken_hex(32))")
  MONGODB_URI         → mongodb://localhost:27017  (or your Atlas URI)
  MONGODB_DB_NAME     → lexis  (default, leave as-is for dev)
  REDIS_URL           → redis://localhost:6379/0   (or your Redis Cloud URL)
  FRONTEND_ORIGIN     → http://localhost:5173       (Vite dev server default)
  ENVIRONMENT         → development

You do NOT need real Firebase/OpenAI keys yet — those are used in later
modules. Leave them empty or with placeholder values for Module 1.

HOW TO START THE SERVER
───────────────────────
  cd backend
  python -m venv venv
  venv\\Scripts\\activate         # Windows
```

```
  pip install -r requirements.txt
  uvicorn main:app --reload
  → http://localhost:8000/api/v1/health
"""

import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from core.config import get_settings
from db.database import connect_mongo, disconnect_mongo
from db.redis_client import connect_redis, disconnect_redis
from utils.logging import setup_logging
from api.routes.health import router as health_router

# ── Logger for this module ──────────────────────────────────────────
logger = logging.getLogger(__name__)


# ── Lifespan: startup & shutdown hooks ──────────────────────────────
@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    Everything BEFORE `yield` runs at startup (once, before any request).
    Everything AFTER `yield` runs at shutdown (once, after last request).

    This is the recommended FastAPI pattern (since v0.93) replacing the
    old on_startup / on_shutdown event decorators.
    """
    settings = get_settings()

    # ── STARTUP ─────────────────────────────────────────────────────
    # 1. Structured JSON logging — must be first so all subsequent logs are JSON
    log_level = "DEBUG" if not settings.is_production else "WARNING"
    setup_logging(log_level=log_level)

    logger.info(
        "LEXIS backend starting up",
        extra={"env": settings.ENVIRONMENT, "version": settings.APP_VERSION},
    )

    # 2. MongoDB
    mongo_client = await connect_mongo(settings.MONGODB_URI, settings.MONGODB_DB_NAME)

    # 3. Redis
    redis_client = await connect_redis(settings.REDIS_URL)

    # 4. Store everything in app.state so routes can access it via request.app.state
    app.state.settings = settings
    app.state.mongo_client = mongo_client
    app.state.mongo_db_name = settings.MONGODB_DB_NAME
    app.state.redis = redis_client

    # 5. Start the Vault Deadline Scheduler (Module 10)
    # APScheduler fires daily at 09:00 to check deadlines and send proactive reminders.
    # If APScheduler is not installed, a warning is logged and the app starts normally.
    from services.vault.scheduler import start_scheduler
    scheduler = start_scheduler(app)
    app.state.scheduler = scheduler

    # 6. Ensure legal knowledge base is seeded in Qdrant (server or in-memory fallback)
    try:
        from db.qdrant_client import get_qdrant_client
        from services.rag.ingestion import seed_starter_knowledge_base
        from agents.rights_agent.agent import get_rights_agent

        q_client = get_qdrant_client()
        count = seed_starter_knowledge_base(client=q_client, force_recreate=False)
        r_agent = get_rights_agent()
        r_agent.qdrant_client = q_client
        logger.info("Qdrant knowledge base initialized/seeded successfully with %d chunks.", cou
```

```
            nt)
    except Exception as exc:
        logger.warning("Could not seed Qdrant knowledge base at startup: %s", exc)

    # 7. Validate Brevo REST API Email Service configuration
    is_brevo_valid, missing_brevo_fields = settings.validate_brevo_config()
    if not is_brevo_valid:
        logger.warning(
            "⚠  [CONFIG] Brevo email service is not fully configured (missing: %s). "
            "Email OTP delivery will fail until valid Brevo credentials are set in backend/.env.",
            ", ".join(missing_brevo_fields),
        )
    elif settings.BREVO_API_KEY.startswith("xsmtpsib-"):
        logger.warning(
            "⚠  [CONFIG] Configured BREVO_API_KEY appears to be an SMTP key (starts with 'xsmtpsib-'). "
            "Brevo REST API requires an API v3 key starting with 'xkeysib-...' from https://app.brevo.com/settings/keys/api."
        )
    else:
        logger.info("Brevo REST API email service configured for sender: %s", settings.BREVO_SENDER_EMAIL)

    logger.info("All connections established. API is ready.")

    # ── Hand control to the running app ──────────────────────────────
    yield

    # ── SHUTDOWN ─────────────────────────────────────────────────────
    logger.info("LEXIS backend shutting down — closing connections…")
    # Shut down the scheduler gracefully before closing DB/Redis connections
    _scheduler = getattr(app.state, "scheduler", None)
    if _scheduler is not None:
        try:
            _scheduler.shutdown(wait=False)
            logger.info("Vault scheduler shut down.")
        except Exception:
            pass
    await disconnect_redis(redis_client)
    await disconnect_mongo(mongo_client)
    logger.info("Shutdown complete.")


# ── App factory ──────────────────────────────────────────────────────
def create_app() -> FastAPI:
    """
    App factory: creates and fully configures a FastAPI instance.

    Returning the app from a function (instead of module-level `app = FastAPI()`)
    makes it trivial to create isolated test instances and to parameterise
    config for different environments.
    """
    settings = get_settings()

    application = FastAPI(
        title=settings.APP_NAME,
        description=(
            "Autonomous Multi-Agent Legal Aid System for Indian Citizens. "
            "Provides rights explanation, document drafting, contract analysis, "
            "scam detection, complaint filing guidance, and document vault."
        ),
        version=settings.APP_VERSION,
        docs_url="/api/docs",        # Swagger UI
        redoc_url="/api/redoc",      # ReDoc UI
        openapi_url="/api/openapi.json",
        lifespan=lifespan,           # Attach our startup/shutdown manager
    )

    # ── CORS — restrict to the configured frontend origin ────────────
    application.add_middleware(
        CORSMiddleware,
        allow_origins=[settings.FRONTEND_ORIGIN],  # Read from .env, never "*"
```

```
        allow_credentials=True,
        allow_methods=["GET", "POST", "PUT", "PATCH", "DELETE", "OPTIONS"],
        allow_headers=["Authorization", "Content-Type", "Accept"],
    )

    # ── Versioned API router ──────────────────────────────────────
    # All routes live under /api/v1 — add future routers here
    from fastapi import APIRouter
    api_v1 = APIRouter(prefix="/api/v1")
    api_v1.include_router(health_router)

    # Module 2: auth router registered under /api/v1/auth/
    from api.routes.auth import router as auth_router
    api_v1.include_router(auth_router)

    # Module 4: query router registered under /api/v1/query/
    from api.routes.query import router as query_router
    api_v1.include_router(query_router)

    # Module 6: drafts router registered under /api/v1/drafts/
    from api.routes.drafts import router as drafts_router
    api_v1.include_router(drafts_router)

    # Module 7: analysis router registered under /api/v1/analysis/
    from api.routes.analysis import router as analysis_router
    api_v1.include_router(analysis_router)

    # Module 8: scam router registered under /api/v1/scam/
    from api.routes.scam import router as scam_router
    api_v1.include_router(scam_router)

    # Module 9: filing guide router registered under /api/v1/filing/
    from api.routes.filing import router as filing_router
    api_v1.include_router(filing_router)

    # Module 10: vault router registered under /api/v1/vault/
    from api.routes.vault import router as vault_router
    api_v1.include_router(vault_router)

    # Module 12: admin router registered under /api/v1/admin/
    from api.routes.admin import router as admin_router
    api_v1.include_router(admin_router)

    # Module 13: lawyer router registered under /api/v1/lawyer/
    # Scaffold only — no functional features. Role guard: require_role(["lawyer"]).
    from api.routes.lawyer import router as lawyer_router
    api_v1.include_router(lawyer_router)

    # Module 13B: users router registered under /api/v1/users/
    from api.routes.users import router as users_router
    api_v1.include_router(users_router)

    application.include_router(api_v1)

    return application


# ── Application instance ──────────────────────────────────────────
# `uvicorn main:app` references this name.
app = create_app()
```

FILE: backend/core/config/settings.py LANG: Python

MODULE: Application Settings, Environment Config & Security Flags

```
"""
LEXIS — Application Settings (Module 1)
----------------------------------------
Pydantic Settings reads every value from the .env file (or environment
variables in production).  Nothing is hardcoded here — if a required
variable is missing, the app fails immediately with a clear error
instead of silently using a wrong default.
"""

from functools import lru_cache
```

```python
from pathlib import Path
from pydantic_settings import BaseSettings, SettingsConfigDict

_BACKEND_DIR = Path(__file__).resolve().parent.parent.parent
_ENV_FILE = _BACKEND_DIR / ".env"


class Settings(BaseSettings):
    """
    All application configuration lives here.
    Every field maps 1-to-1 to a line in .env / environment variables.
    """

    # ── App ──────────────────────────────────────────────────────────
    APP_NAME: str = "LEXIS API"
    APP_VERSION: str = "1.0.0"
    ENVIRONMENT: str = "development"     # "development" | "production"
    SECRET_KEY: str                       # Required — JWT / session signing

    # ── MongoDB (Motor async driver) ─────────────────────────────────
    MONGODB_URI: str                      # e.g. mongodb://localhost:27017
    MONGODB_DB_NAME: str = "lexis"        # Database name inside Mongo

    # ── Redis ────────────────────────────────────────────────────────
    REDIS_URL: str                        # e.g. redis://localhost:6379/0

    # ── JWT ──────────────────────────────────────────────────────────
    JWT_ACCESS_EXPIRE_MINUTES: int = 15   # Access token lifetime (15 min)
    JWT_REFRESH_EXPIRE_DAYS: int = 7      # Refresh token lifetime (7 days)

    # ── OTP ──────────────────────────────────────────────────────────
    OTP_TTL_SECONDS: int = 300            # OTP valid for 5 minutes
    OTP_MAX_ATTEMPTS: int = 3             # Max wrong guesses before OTP is invalidated
    OTP_RATE_LIMIT_MAX: int = 3           # Max send-otp calls per window per phone
    OTP_RATE_LIMIT_WINDOW_SECONDS: int = 60  # Rate limit window duration

    # ── CORS — frontend origin (never use "*") ──────────────────────
    FRONTEND_ORIGIN: str = "http://localhost:5173"

    # ── LLM (Gemini) ─────────────────────────────────────────────────
    GEMINI_API_KEY: str = ""
    GEMINI_MODEL_DEV: str = "gemini-3.6-flash"
    OPENAI_API_KEY: str = ""
    OPENAI_MODEL_DEV: str = "gpt-4o-mini"
    OPENAI_MODEL_DEMO: str = "gpt-4o"

    # ── Firebase Auth ────────────────────────────────────────────────
    FIREBASE_PROJECT_ID: str = ""
    FIREBASE_PRIVATE_KEY: str = ""
    FIREBASE_CLIENT_EMAIL: str = ""
    FIREBASE_SERVICE_ACCOUNT_PATH: str = "firebase/service_account.json"

    # ── Qdrant Vector DB (Module 5) ──────────────────────────────────
    QDRANT_HOST: str = "localhost"
    QDRANT_PORT: int = 6333
    QDRANT_COLLECTION_NAME: str = "legal_knowledge_base"
    QDRANT_API_KEY: str = ""
    QDRANT_STORAGE_PATH: str = "./qdrant_storage"

    # ── Scam Detection (Module 8) ────────────────────────────────────
    # Separate Qdrant collection for scam pattern embeddings.
    # MongoDB collection `scam_patterns` is the authoritative source for alert content.
    SCAM_COLLECTION_NAME: str = "scam_patterns"

    # ── Filing Guide (Module 9) ──────────────────────────────────────
    # MongoDB collection storing the verified filing authority directory.
    # Seeded from backend/data/filing_authorities/filing_seed.json.
    # No Qdrant collection — exact-match lookup by doc_type only (no semantic search).
    FILING_COLLECTION_NAME: str = "filing_authorities"

    # ── Vault / Storage (Module 7 + Module 10) ──────────────────────
    VAULT_STORAGE_PATH: str = "./vault_storage"
    UPLOAD_MAX_SIZE_MB: int = 10        # Max uploaded file size for Document Analyzer
```

```
    # ── Vault Encryption (Module 10) ──────────────────────────────────
    # AES-256-GCM key for encrypting vault documents at rest.
    # Must be a base64-encoded 32-byte (256-bit) random key.
    # Generate: python -c "import secrets,base64; print(base64.b64encode(secrets.token_bytes(32)
        ).decode())"
    # WHY NOT DEFERRED TO MODULE 14: Vault stores passports/Aadhaar. Data written
    # today is stored today. Deferring encryption to Module 14 means sensitive
    # documents sit as plaintext on disk for 4 modules — unacceptable.
    # Module 14 will VERIFY this encryption is in place, not enable it.
    VAULT_ENCRYPTION_KEY: str = ""      # Required for vault uploads; app warns if empty
    VAULT_COLLECTION_NAME: str = "vault_documents"
    DEADLINE_COLLECTION_NAME: str = "vault_deadlines"

    # -- Memory & Reflection (Module 11) ----------------------------------------
    # MongoDB collection for Confirmation Agent reflection pass logs.
    # Queryable by the Admin Dashboard (Module 12) for quality analytics.
    REFLECTION_LOG_COLLECTION: str = "reflection_logs"
    # Redis session TTL in seconds (default: 24 hours).
    # Sessions older than this are automatically expired from Redis.
    SESSION_TTL_SECONDS: int = 86_400

    # ── Email Notifications & Brevo REST API ──────────────────────────
    # Brevo (formerly Sendinblue) v3 REST API for transactional emails & Email OTP.
    # Uses https://api.brevo.com/v3/smtp/email with api-key header.
    BREVO_API_KEY: str = ""
    BREVO_SENDER_EMAIL: str = "noreply@lexis.local"
    BREVO_SENDER_NAME: str = "LEXIS"

    # ── Scheduling ────────────────────────────────────────────────────
    NOTIFICATION_CHECK_INTERVAL_MINUTES: int = 60

    # Pydantic v2 config: look for a .env file in the backend directory
    model_config = SettingsConfigDict(
        env_file=(_ENV_FILE, ".env"),
        env_file_encoding="utf-8",
        case_sensitive=True,
        extra="ignore",  # silently ignore any unrecognised env vars
    )

    @property
    def is_production(self) -> bool:
        """True when running in a production environment."""
        return self.ENVIRONMENT.lower() == "production"

    def validate_brevo_config(self) -> tuple[bool, list[str]]:
        """
        Validates Brevo configuration at startup or service initialization.
        Returns (is_valid, list_of_missing_fields).
        Never exposes the API key.
        """
        missing = []
        if not (self.BREVO_API_KEY or "").strip():
            missing.append("BREVO_API_KEY")
        if not (self.BREVO_SENDER_EMAIL or "").strip():
            missing.append("BREVO_SENDER_EMAIL")
        if not (self.BREVO_SENDER_NAME or "").strip():
            missing.append("BREVO_SENDER_NAME")
        return len(missing) == 0, missing


@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """
    Returns a cached Settings singleton.
    Using lru_cache means .env is parsed exactly once at startup —
    subsequent calls get the same object instantly.
    """
    return Settings()
```

FILE: backend/agents/orchestrator/state.py LANG: Python

MODULE: LangGraph Multi-Agent Orchestrator State & Reducers

```
"""
```

```
╔══════════════════════════════════════════════════════════════════╗
║          LEXIS Orchestrator — Shared State Schema (Module 4)     ║
╚══════════════════════════════════════════════════════════════════╝

WHAT IS A LANGGRAPH STATE AND WHY DO WE NEED IT?
─────────────────────────────────────────────────
A LangGraph graph is a state machine.  A state machine is a program that:

  1. Holds a snapshot of all relevant data (the "state").
  2. Passes that snapshot through a series of processing steps (the "nodes").
  3. Each node reads what it needs from the state and writes its result back.
  4. A special rule ("edge") decides which node runs next.

Without a shared state, every node would have to pass data to the next node
explicitly — like passing a shopping bag from person to person where each
person can only see what the person before them put in.

With a shared state, every node reads from and writes to one central object,
like a whiteboard that everyone in the room can see and write on.  This is
why the same state is "shared across agents".

WHY NOT JUST USE if/else IN ONE FUNCTION?
──────────────────────────────────────────
If we classified the intent and then called each agent inside one big Python
function, the agents would run one after the other (sequentially).  The
second agent would only start after the first finished.  For 3 agents on one
query, the user would wait 3× as long.

A graph lets us ROUTE: after classification, we send the state to multiple
nodes that run at the SAME TIME (concurrently).  The user waits for the
slowest agent, not the sum of all agents.  This is not "just calling an LLM"
— it is orchestrating multiple LLM calls concurrently, with controlled state,
failure isolation, and a defined path to compile the results.

HOW CONCURRENT NODE EXECUTION DIFFERS FROM SEQUENTIAL
─────────────────────────────────────────────────────
Sequential (bad):  classify → agentA (3 s) → agentB (3 s) → agentC (3 s) = 9 s total
Concurrent (good): classify → agentA ┐
                            → agentB ├─ all run at once ─→ compile = ~3 s total
                            → agentC ┘

LangGraph achieves this via the Send API: a "router" function returns a list
of Send objects, each specifying a target node.  LangGraph schedules all of
them in the SAME super-step, which means they run in parallel.

WHY REDUCERS ARE REQUIRED FOR CONCURRENT WRITES
────────────────────────────────────────────────
If two nodes try to write to the same state key at the same time, the last
one to finish would overwrite the first — you would lose data.  A "reducer"
is a function that tells LangGraph HOW to merge multiple writes to the same
key.

  Annotated[List[AgentOutput], operator.add]

means "when multiple nodes write to `scratchpad`, append all their lists
together" — like appending to a shared list, safe under any ordering.
"""

import operator
from enum import Enum
from typing import Annotated, Any, Dict, List, Optional

from typing_extensions import TypedDict

# ── Reducer for retry_counts dict ──────────────────────────────────────
# When multiple parallel nodes return retry_counts updates, we need to MERGE
# the dicts (not overwrite).  operator.add works for lists but not dicts.
# We define a custom reducer: merge two dicts, taking the MAX value per key.
# This is safe under concurrent writes: if two parallel nodes both increment
# the retry counter for the same intent, the higher value wins (conservative).
def _merge_retry_counts(a: Dict[str, int], b: Dict[str, int]) -> Dict[str, int]:
    merged = dict(a)
    for k, v in b.items():
```

```
        merged[k] = max(merged.get(k, 0), v)
    return merged


# ── Intent categories ──────────────────────────────────────────────

class AgentIntent(str, Enum):
    """
    The six specialist agent categories that the orchestrator can route to.

    A single citizen query can match MORE THAN ONE intent.  For example:
    "My landlord is not returning my deposit and I want to file a complaint"
    matches both RIGHTS (tenant rights) and FILING (complaint filing guide).
    When that happens, both specialist nodes run concurrently.
    """
    RIGHTS  = "rights"    # Explain legal rights (Module 5)
    DRAFT   = "draft"     # Draft a legal document (Module 6)
    ANALYZE = "analyze"   # Analyze a document for risk (Module 7)
    SCAM    = "scam"      # Detect and explain a scam (Module 8)
    FILING  = "filing"    # Guide on where/how to file a complaint (Module 9)
    VAULT   = "vault"     # Vault & deadline management (Module 10)


# ── Per-agent output record ────────────────────────────────────────

class AgentOutput(TypedDict):
    """
    One record written by a specialist node into the shared scratchpad.

    Each specialist node produces exactly ONE AgentOutput.  All of them are
    collected in the `scratchpad` list, which is then read by the compiler
    node to assemble the final response.

    `status` values:
      "ok"      — agent completed successfully
      "error"   — agent threw an exception (partial result: still included)
      "timeout" — agent hit its wall-clock limit (partial result: included)
    """
    intent:   str               # Which AgentIntent this result came from
    status:   str               # "ok" | "error" | "timeout"
    output:   str               # The agent's prose answer (or error description)
    metadata: Dict[str, Any]    # Extra info: latency_ms, model used, etc.


# ── Full orchestrator state ────────────────────────────────────────

class OrchestratorState(TypedDict):
    """
    The single shared state object that flows through every node in the graph.

    ┌──────────────────────────┬──────────────────────────────────────────┐
    │  FIELD                   │  WHO WRITES IT                           │
    ├──────────────────────────┼──────────────────────────────────────────┤
    │  query                   │  API layer (initial state)               │
    │  session_id              │  API layer (random UUID per request)     │
    │  conversation_history    │  API layer + memory module (M11)         │
    │  intents                 │  classify_intent_node                    │
    │  confidence              │  classify_intent_node                    │
    │  scratchpad              │  each specialist_node (via reducer)      │
    │  final_response          │  compile_response_node                   │
    │  status                  │  compile_response_node                   │
    │  llm_calls_used          │  classify + each specialist node         │
    │  llm_calls_limit         │  API layer (initial state)               │
    │  retry_target            │  Module 11 Confirmation Agent            │
    │  retry_counts            │  Module 11 Confirmation Agent            │
    │  retry_feedback          │  Module 11 Confirmation Agent            │
    │  contradiction_note      │  Module 11 Confirmation Agent            │
    └──────────────────────────┴──────────────────────────────────────────┘
    """

    # ── Input ──────────────────────────────────────────────────────
    query: str
    """The raw citizen query string, as received from the frontend."""
```

```
    session_id: str
    """
    Random UUID identifying this conversation session.
    Used by the Session Memory Store (Module 11) to key Redis hashes.
    NOT derived from the user's phone or MongoDB _id — generated fresh
    per request via uuid4().
    """

    conversation_history: List[Dict[str, str]]
    """
    Prior turns in this session (list of {"role": "user"|"assistant", "content": "..."}).
    Passed to the LLM for context.  Empty list for first-turn queries.
    """

    # ── Classification outputs ──────────────────────────────────────
    intents: List[AgentIntent]
    """
    Detected intents for this query (empty list = unclear, 1+ = route to those agents).
    Populated by classify_intent_node.  The fan_out_router reads this list
    to decide how many Send objects to create (one per intent).
    """

    confidence: float
    """
    Classifier confidence in [0.0, 1.0].  Below 0.4 → status = "unclear"
    even if intents were detected.
    """

    # ── Concurrent agent outputs (reducer = append-safe under parallelism) ─
    scratchpad: Annotated[List[AgentOutput], operator.add]
    """
    Outputs from all specialist nodes.  Each specialist appends ONE AgentOutput
    to this list.  The `operator.add` reducer means LangGraph APPENDS new
    entries instead of overwriting, making this safe when multiple agents
    run concurrently.

    This is the "shared scratchpad state across agents" from the project spec.
    """

    # ── Compiled result ─────────────────────────────────────────────
    final_response: str
    """Human-readable, compiled response from all agent outputs."""

    status: str
    """
    Overall status of the request:
      "ok"      — all matched agents succeeded
      "partial" — at least one agent succeeded, at least one failed/timed out
      "unclear" — classifier could not determine intent with enough confidence
      "error"   — all agents failed or no agents were dispatched
    """

    # ── Cost / latency guardrails ───────────────────────────────────
    llm_calls_used: Annotated[int, operator.add]
    """
    Running count of LLM calls made during this request (classifier + all agents).

    Uses operator.add as a reducer so that when multiple specialist nodes run
    concurrently and each increments this counter, LangGraph SUMS all the
    increments instead of raising InvalidUpdateError for concurrent writes.
    """

    llm_calls_limit: int
    """
    Maximum LLM calls allowed per request.  Set by the API layer from settings.
    The compile_response_node checks this and stops dispatching if exceeded.
    Logged as a WARNING if any request approaches or exceeds this ceiling.
    """

    # ── Module 11: Retry / Loopback ─────────────────────────────────
    retry_target: Optional[AgentIntent]
    """
```

```
    Set by the Confirmation Agent when an agent's output fails validation
    and has retries remaining.  compile_response_node routes back to
    specialist_node for this intent.

    None when no retry is needed (all agents passed, or cap reached).
    """

    retry_counts: Annotated[Dict[str, int], _merge_retry_counts]
    """
    Tracks how many times each specialist agent has been retried on this
    request.  Key = AgentIntent value string (e.g. "rights").
    Value = number of COMPLETED retries (0 = never retried).

    The Confirmation Agent checks this before requesting a retry:
        if retry_counts.get(intent, 0) >= MAX_RETRIES:
            → return honest fallback instead of routing back

    Uses _merge_retry_counts reducer (takes MAX per key) to be safe
    under concurrent writes from parallel agent nodes.
    """

    retry_feedback: str
    """
    Actionable feedback message sent back to the specialist agent on retry.
    Populated by the Confirmation Agent with the specific failure reason and
    what the agent must produce to pass.

    Empty string when no retry is needed.
    """

    contradiction_note: Optional[str]
    """
    Advisory note appended to the final response when the Confirmation Agent
    detects a cross-agent contradiction (e.g. Rights Agent and Filing Guide
    citing different governing Acts).

    None when no contradiction is detected.
    The note tells the citizen that two responses may be inconsistent and
    advises them to consult a legal professional.
    """

    confirmed_outputs: Optional[List[AgentOutput]]
    """
    The Confirmation Agent writes the validated (and possibly fallback-wrapped)
    agent outputs here instead of back to scratchpad.

    WHY A SEPARATE FIELD:
    scratchpad uses operator.add reducer -- writing back to it from the
    confirmation node would APPEND the approved outputs to the EXISTING
    scratchpad, doubling the entries.  confirmed_outputs has no reducer (plain
    replacement), so it safely overwrites.

    compile_response_node reads confirmed_outputs if non-None, otherwise
    falls back to scratchpad (for tests or code paths that bypass confirmation).
    """
```

FILE: backend/agents/orchestrator/agent.py LANG: Python

MODULE: LangGraph State Machine, Intent Classifier & Router

```
"""
================================================================================
LEXIS Orchestrator Agent -- Module 4: Orchestrator Agent Core
================================================================================

WHAT IS THIS FILE?
------------------
This file defines the LEXIS Orchestrator: a LangGraph state machine that
receives a citizen's legal query, classifies it into one or more intent
categories, dispatches specialist agent nodes CONCURRENTLY for each matched
intent, collects and merges their outputs safely, and compiles a single
unified response.

WHAT IS A LANGGRAPH GRAPH AND WHY NOT JUST USE if/else?
-------------------------------------------------------
```

FILE: backend/services/vault/scheduler.py LANG: Python
MODULE: Proactive Vault Deadline APScheduler Daily Automation

```
"""
================================================================================
LEXIS — Vault Deadline Scheduler (Module 10)
================================================================================

HOW THE SCHEDULER WORKS (PLAIN LANGUAGE FOR BEGINNERS)
------------------------------------------------------
This is the component that makes the Vault & Deadline Agent "proactive" —
it acts without the citizen asking again.

We use APScheduler (AsyncIOScheduler) to run a cron job once per day at 9 AM.
Each run does the following:

  1. Query ALL deadline records in the database that fall within the next 31 days.
  2. For each deadline:
       a. Compute: days_remaining = deadline_date - today
       b. Check: is days_remaining in [30, 15, 3]?
       c. Check: has this interval already been sent? (reminders_sent list)
       d. If both conditions are YES: fire the reminder.
  3. Firing a reminder means:
       a. Call VaultDeadlineAgent.invoke_document_drafter() to generate a
          proactive draft (lease renewal letter, visa extension inquiry, etc.)
       b. Send the citizen an email (via NotificationService) with the draft attached.
       c. Update the deadline record: add the interval to reminders_sent.

WHY T-30, T-15, T-3 (NOT DAILY)?
---------------------------------
Daily emails would be spam and train citizens to ignore them. Three targeted
reminders at meaningful intervals:
  - T-30 days: "You have a month — start thinking about this"
  - T-15 days: "Two weeks — start preparing"
  - T-3 days:  "Urgent — act now"

Each reminder fires exactly ONCE per interval because the interval is recorded
in reminders_sent immediately after firing. Even if the scheduler runs daily,
the check `interval not in record.reminders_sent` prevents duplicate sends.

WHY APSCHEDULER (NOT CELERY/BULLMQ)?
-------------------------------------
APScheduler 3.x runs inside the same Python process as FastAPI via AsyncIOScheduler.
This means:
  - No separate worker process to deploy
  - No Redis queue required (we already use Redis for auth, but the scheduler
    doesn't need it for its own operation)
  - The scheduler shares the app's MongoDB connection pool
  - Trivial to start: one function call in the lifespan hook

For production scale (millions of users), a distributed job queue (Celery + Redis,
or BullMQ) would be better. For LEXIS MVP, APScheduler is appropriate.
"""

import asyncio
import logging
from datetime import date, datetime, timedelta
from typing import Optional

logger = logging.getLogger(__name__)

# ── Reminder intervals (days before deadline) ─────────────────────────────
# These are the ONLY intervals that trigger a reminder.
# Any other days_remaining value is silently skipped.
REMINDER_INTERVALS = [30, 15, 3]


async def daily_deadline_check(db) -> dict:
    """
    Core scheduler job: checks all upcoming deadlines and fires reminders.

    This function is designed to be testable in isolation:
      - It accepts `db` as a parameter (not from app.state) so unit tests
```

```
        can pass a mock DB.
      - It accepts an optional `today` override for testing without waiting real days.
      - It returns a summary dict (fired, skipped, errors) so tests can assert behavior.

    Parameters:
        db    — Motor async MongoDB database handle
        today — Override for today's date (for testing). Defaults to date.today().

    Returns:
        {
            "fired": int,   — number of reminders sent this run
            "skipped": int, — deadlines checked but not at a trigger interval
            "errors": int,  — deadlines that caused exceptions
        }
    """
    today = date.today()

    from services.vault.deadline_service import get_deadline_service
    deadline_svc = get_deadline_service()

    records = await deadline_svc.get_all_upcoming_deadlines(db, days_ahead=31)

    fired = 0
    skipped = 0
    errors = 0

    for record in records:
        try:
            # Compute days remaining
            deadline_dt = datetime.strptime(record.deadline_date, "%Y-%m-%d").date()
            days_remaining = (deadline_dt - today).days

            # Only fire at the configured intervals
            if days_remaining not in REMINDER_INTERVALS:
                skipped += 1
                continue

            # Check if this interval was already sent
            if days_remaining in record.reminders_sent:
                logger.debug(
                    "Reminder already sent for this interval",
                    extra={
                        "deadline_id": record.deadline_id,
                        "interval": days_remaining,
                    },
                )
                skipped += 1
                continue

            # ── Fire the reminder ──────────────────────────────────────
            logger.info(
                "Firing deadline reminder",
                extra={
                    "deadline_id": record.deadline_id,
                    "user_id": record.user_id,
                    "label": record.label,
                    "days_remaining": days_remaining,
                },
            )

            # Step 1: Invoke Document Drafter to generate a proactive draft
            # This is the "agentic" part — the system drafts a document on
            # the citizen's behalf WITHOUT them asking.
            draft_text = ""
            try:
                from agents.vault_deadline.agent import get_vault_agent
                vault_agent = get_vault_agent()
                draft_text = await vault_agent.invoke_document_drafter(
                    doc_type=record.doc_type,
                    context={
                        "label": record.label,
                        "deadline_date": record.deadline_date,
                        "days_remaining": days_remaining,
```

```
                        "user_id": record.user_id,
                    },
                )
            except Exception as draft_exc:
                logger.warning(
                    "Document Drafter invocation failed during reminder (non-fatal): %s",
                    draft_exc,
                )

            # Step 2: Send email notification
            try:
                from services.notifications.notification_service import get_notification_service
                notif_svc = get_notification_service()
                await notif_svc.send_deadline_notification(
                    user_id=record.user_id,
                    deadline_label=record.label,
                    deadline_date=record.deadline_date,
                    days_remaining=days_remaining,
                    draft_text=draft_text,
                    db=db,
                )
            except Exception as notif_exc:
                logger.warning(
                    "Notification send failed during reminder (non-fatal): %s",
                    notif_exc,
                )

            # Step 3: Mark this interval as sent (prevent duplicates)
            await deadline_svc.mark_reminder_sent(
                db=db,
                deadline_id=record.deadline_id,
                interval=days_remaining,
                draft_text=draft_text if draft_text else None,
            )

            fired += 1

        except Exception as exc:
            logger.error(
                "Error processing deadline record",
                extra={"deadline_id": record.deadline_id, "error": str(exc)},
                exc_info=True,
            )
            errors += 1

    logger.info(
        "Daily deadline check complete",
        extra={"fired": fired, "skipped": skipped, "errors": errors, "date": today.isoformat()},
    )

    return {"fired": fired, "skipped": skipped, "errors": errors}


def start_scheduler(app) -> Optional[object]:
    """
    Creates and starts the APScheduler AsyncIOScheduler.

    Called from main.py's lifespan hook after MongoDB is connected.
    The scheduler runs inside the same asyncio event loop as FastAPI.

    Returns the scheduler object (so it can be shut down in the lifespan teardown).
    Returns None if APScheduler is not installed (graceful degradation).

    SCHEDULE: every day at 09:00 server-local time.
    To change the schedule, edit the cron parameters below.
    """
    try:
        from apscheduler.schedulers.asyncio import AsyncIOScheduler
        from apscheduler.triggers.cron import CronTrigger
    except ImportError:
        logger.warning(
            "APScheduler not installed — deadline reminders will not run. "
            "Install: pip install apscheduler==3.10.4"
```

```
        )
        return None

    scheduler = AsyncIOScheduler()

    async def _scheduled_job():
        """Wrapper that injects the DB from app.state into daily_deadline_check."""
        try:
            db = app.state.mongo_client[app.state.mongo_db_name]
            await daily_deadline_check(db=db)
        except Exception as exc:
            logger.error("Scheduler job failed: %s", exc, exc_info=True)

    scheduler.add_job(
        _scheduled_job,
        trigger=CronTrigger(hour=9, minute=0),   # 9:00 AM daily
        id="daily_deadline_check",
        name="LEXIS Vault Deadline Reminder Check",
        replace_existing=True,
        misfire_grace_time=3600,  # If missed, run within 1 hour
    )

    scheduler.start()
    logger.info(
        "Vault deadline scheduler started — daily job at 09:00",
        extra={"job_id": "daily_deadline_check"},
    )

    return scheduler
```

FILE: frontend/src/App.jsx LANG: JavaScript (React JSX)

MODULE: Frontend Root Component, Routes & Protected Guards

```
/*
 * frontend/src/App.jsx
 * ─────────────────────────────────────────────────────────────────────
 * LEXIS — Route Definitions (Module 3)
 *
 * HOW ROUTING WORKS IN THIS APP:
 *
 *   React Router v6 uses a component-based routing system.
 *   You declare <Route> elements here — each one maps a URL path to a React
 *   component.  When the user navigates to a URL, React Router looks for a
 *   matching <Route> and renders its component.
 *
 *   <Routes> — a container; only ONE child route matches at a time.
 *   <Route path="..." element={<Component />}> — when the URL matches "path",
 *     React renders <Component> in place of the <Routes> block.
 *
 * NESTED ROUTES (how layout + page share the screen):
 *
 *   We have a layout component (AppLayout) that provides the header/sidebar/footer.
 *   By nesting page routes INSIDE the layout route, each page automatically
 *   gets the shared layout.  AppLayout uses <Outlet /> as a placeholder —
 *   React Router replaces it with whichever nested route matched.
 *
 *   Example:
 *     URL "/"  →  AppLayout renders, <Outlet> becomes <Home>
 *     URL "/rights-explainer"  →  AppLayout renders, <Outlet> becomes <RightsExplainer>
 *
 * PROTECTED ROUTES:
 *
 *   <ProtectedRoute> is a wrapper component.  It checks if the user is
 *   authenticated.  If NOT → redirect to /login.  If YES → render <Outlet>
 *   (which is the nested AppLayout + page).
 *
 *   So the guard chain is:
 *     ProtectedRoute  → checks auth → AppLayout → checks URL → Page component
 * ─────────────────────────────────────────────────────────────────────
 */

import React from 'react';
import { Routes, Route, Navigate } from 'react-router-dom';

```

```
// ── Layout & guard ──────────────────────────────────────────────────
import ProtectedRoute from './components/ProtectedRoute';
import AppLayout from './components/layout/AppLayout';

// ── Pages ───────────────────────────────────────────────────────────
import Login            from './pages/Login';
import Home             from './pages/Home';
import RightsExplainer  from './pages/RightsExplainer';
import DocumentDrafter  from './pages/DocumentDrafter';
import DocumentAnalyzer from './pages/DocumentAnalyzer';
import ScamDetection    from './pages/ScamDetection';
import FilingGuide      from './pages/FilingGuide';
import VaultDeadlines   from './pages/VaultDeadlines';
import AdminDashboard   from './pages/AdminDashboard';
import LawyerDashboard from './pages/LawyerDashboard';  // Module 13 scaffold
import ProfileSetup    from './pages/ProfileSetup';     // Module 13B
import Settings        from './pages/Settings';         // Module 13B


function App() {
  return (
    <Routes>

      {/* ── Public routes (no auth required) ──────────────────────── */}
      <Route path="/login" element={<Login />} />

      {/* ── Protected routes (auth required) ──────────────────────── */}
      {/*
        <ProtectedRoute> wraps all authenticated pages.
        It has no path of its own — it just acts as a guard.
        If the user is NOT logged in, they are redirected to /login.
        If they ARE logged in, <Outlet> renders, which is <AppLayout>.
        <AppLayout> then renders another <Outlet> for the specific page.
      */}
      <Route element={<ProtectedRoute />}>
        <Route element={<AppLayout />}>
          <Route index element={<Home />} />                                    {/* / */}
          <Route path="rights-explainer"  element={<RightsExplainer />} />    {/* /rights-explai
              ner */}
          <Route path="document-drafter"  element={<DocumentDrafter />} />    {/* /document-draf
              ter */}
          <Route path="document-analyzer" element={<DocumentAnalyzer />} />   {/* /document-anal
              yzer */}
          <Route path="scam-detection"    element={<ScamDetection />} />      {/* /scam-detectio
              n */}
          <Route path="filing-guide"      element={<FilingGuide />} />        {/* /filing-guide
              */}
          <Route path="vault-deadlines"   element={<VaultDeadlines />} />     {/* /vault-deadlin
              es */}
          <Route path="profile-setup"     element={<ProfileSetup />} />        {/* /profile-setu
              p */}
          <Route path="settings"          element={<Settings />} />            {/* /settings */}
        </Route>
      </Route>

      {/* ── Admin-only protected route (allowedRoles=['admin']) ──────── */}
      <Route element={<ProtectedRoute allowedRoles={['admin']} />}>
        <Route element={<AppLayout />}>
          <Route path="admin" element={<AdminDashboard />} />                {/* /admin */}
        </Route>
      </Route>

      {/* ── Lawyer-only protected route (allowedRoles=['lawyer']) ──────── */}
      {/*
        Module 13 scaffold: lawyer role has its own isolated route.
        admin, user, and anonymous JWTs are redirected to home (/).
        No functional features — Coming Soon screen only.
      */}
      <Route element={<ProtectedRoute allowedRoles={['lawyer']} />}>
        <Route element={<AppLayout />}>
          <Route path="lawyer" element={<LawyerDashboard />} />           {/* /lawyer */}
        </Route>
      </Route>
```

```
      {/* ── Catch-all: unknown URLs go to dashboard (or login if not authed) */}
      <Route path="*" element={<Navigate to="/" replace />} />

    </Routes>
  );
}

export default App;
```

FILE: frontend/src/context/AuthContext.jsx LANG: JavaScript (React JSX)

MODULE: Authentication Context, JWT Lifecycle & Session State

```
/*
 * frontend/src/context/AuthContext.jsx
 * ─────────────────────────────────────────────────────────────────────────
 * LEXIS — Authentication Context (Module 3)
 *
 * WHAT IS "CONTEXT" IN REACT?
 *   Normally, to share data between components you have to pass it as "props"
 *   down through every level of the component tree.  Context is a shortcut:
 *   you wrap your whole app in a "Provider", and any component — no matter
 *   how deep — can read the shared data without prop-drilling.
 *
 * WHAT DOES THIS CONTEXT STORE?
 *   - user        : the user object returned by the backend (phone, role, id)
 *   - accessToken : the short-lived JWT (15 min) used on every request
 *   - refreshToken: the long-lived JWT (7 days) used to get a new access token
 *   - isLoading   : true while we're checking localStorage at startup
 *
 * HOW LOGIN / LOGOUT WORK:
 *   login(tokens, userObj) → saves tokens to localStorage AND to React state
 *   logout()               → clears localStorage AND React state, goes to /login
 *
 * WHY BOTH LOCALSTORAGE AND STATE?
 *   React state resets when the page refreshes.  localStorage persists across
 *   refreshes.  On startup, we read from localStorage to restore the session,
 *   then keep state in sync for the rest of the browser session.
 * ─────────────────────────────────────────────────────────────────────────
 */

import React, { createContext, useContext, useEffect, useState } from 'react';
import { useNavigate } from 'react-router-dom';

// ── Create the context object ──────────────────────────────────────────
// createContext(defaultValue) — the default is only used if a component is
// rendered outside of the Provider (which shouldn't happen in our app).
const AuthContext = createContext(null);


// ── AuthProvider component ─────────────────────────────────────────────
// This wraps the entire app (see main.jsx).  It stores auth state and
// exposes login/logout functions to every component inside it.
export function AuthProvider({ children }) {
  // State: what we know about the currently logged-in user.
  const [user, setUser]               = useState(null);
  const [accessToken, setAccessToken] = useState(null);
  const [refreshToken, setRefreshToken] = useState(null);

  // isLoading is true only for the first render while we read localStorage.
  // We show nothing (or a spinner) until this is false, to avoid a flash
  // where the app briefly thinks the user is logged out.
  const [isLoading, setIsLoading]     = useState(true);

  const navigate = useNavigate();  // React Router hook — lets us change pages programmatically

  // ── Restore session from localStorage on first load ──────────────────
  // useEffect with an empty dependency array [] runs exactly ONCE,
  // right after the component mounts (appears on screen for the first time).
  useEffect(() => {
    const storedAccess  = localStorage.getItem('lexis_access_token');
    const storedRefresh = localStorage.getItem('lexis_refresh_token');
    const storedUser    = localStorage.getItem('lexis_user');

```

```
  if (storedAccess && storedUser) {
    // We have a saved session — restore it into React state.
    try {
      setUser(JSON.parse(storedUser));      // localStorage stores strings; parse back to obje
          ct
      setAccessToken(storedAccess);
      setRefreshToken(storedRefresh);
    } catch {
      // If parsing fails (corrupted data), clear everything and start fresh.
      clearStorage();
    }
  }

  // Done loading — the app can now decide which route to show.
  setIsLoading(false);
}, []); // [] means "only run once on mount"


// ── login() — called after successful OTP verification ──────────────────
// Parameters:
//   tokens  : { access_token, refresh_token } from the backend response
//   userObj : the user profile we got from /auth/me (or we build a minimal one)
function login(tokens, userObj) {
  const { access_token, refresh_token } = tokens;

  // 1. Save to localStorage so the session survives a page refresh.
  localStorage.setItem('lexis_access_token',  access_token);
  localStorage.setItem('lexis_refresh_token', refresh_token);
  localStorage.setItem('lexis_user',          JSON.stringify(userObj));

  // 2. Update React state so the rest of the app re-renders with the new user.
  setAccessToken(access_token);
  setRefreshToken(refresh_token);
  setUser(userObj);

  // 3. Navigate to the dashboard.
  navigate('/');
}


// ── logout() — called when the user clicks "Sign out" or on 401 ─────────
function logout() {
  clearStorage();

  // Reset state — every component that reads from context will re-render.
  setUser(null);
  setAccessToken(null);
  setRefreshToken(null);

  // Go back to the login page.
  navigate('/login');
}


// ── clearStorage() — helper to remove all auth keys ─────────────────────
function clearStorage() {
  localStorage.removeItem('lexis_access_token');
  localStorage.removeItem('lexis_refresh_token');
  localStorage.removeItem('lexis_user');
}


// ── Convenience getter ──────────────────────────────────────────────────
// True when we have an access token (i.e. user is logged in).
const isAuthenticated = Boolean(accessToken);


// ── What we expose to every component that uses useAuth() ───────────────
const value = {
  user,           // The user object { id, phone, role, ... }
  accessToken,    // The JWT access token string (15-min lifetime)
  refreshToken,   // The JWT refresh token string (7-day lifetime)
  isLoading,      // True while we're reading localStorage on first load
```

```
    isAuthenticated,// Boolean shortcut — true if logged in
    login,          // Call after verifyOtp succeeds
    logout,         // Call on sign-out or 401
  };

  return (
    // The Provider makes `value` available to every child component.
    <AuthContext.Provider value={value}>
      {children}
    </AuthContext.Provider>
  );
}


// ── useAuth() hook ──────────────────────────────────────────────────
// Any component can call:  const { user, login, logout } = useAuth();
// This is cleaner than importing AuthContext directly everywhere.
export function useAuth() {
  const context = useContext(AuthContext);

  // If context is null, the component is used outside of AuthProvider.
  // This is a programming mistake — throw a clear error instead of a cryptic one.
  if (context === null) {
    throw new Error('useAuth() must be used inside <AuthProvider>. Check main.jsx.');
  }

  return context;
}

export default AuthContext;
```

FILE: frontend/src/services/api.js LANG: JavaScript

MODULE: Axios API Client Layer with Automated JWT Interceptors

```
/*
 * frontend/src/services/api.js
 * ─────────────────────────────────────────────────────────────────────
 * LEXIS — Axios API Client (Module 3)
 *
 * WHAT IS THIS FILE?
 *   Axios is a JavaScript library that makes HTTP requests (GET, POST, etc.)
 *   to the backend.  Instead of writing the base URL in every request, we
 *   create ONE "instance" here with default settings.  Every other file just
 *   imports this instance and calls it directly.
 *
 * HOW DOES THE TOKEN GET ATTACHED AUTOMATICALLY?
 *   An "interceptor" is a function that runs automatically on every request
 *   BEFORE it is sent (or on every response AFTER it is received).
 *
 *   Request interceptor (lines below):
 *     1. Reads the access token from localStorage.
 *     2. If a token exists, it adds the header:
 *        Authorization: Bearer <token>
 *     The backend's get_current_user dependency reads that header to identify
 *     who is making the request.
 *
 *   Response interceptor:
 *     1. If the backend replies with HTTP 401 (Unauthorized — usually "token
 *        expired or missing"), we clear the stored tokens and redirect the
 *        browser to /login so the user can sign in again.
 *
 * VITE ENV VARIABLE:
 *   import.meta.env.VITE_API_BASE_URL comes from frontend/.env.
 *   Vite automatically injects variables that start with VITE_ into the app.
 *   The fallback 'http://localhost:8000' is a safe default for local dev.
 * ─────────────────────────────────────────────────────────────────────
 */

import axios from 'axios';

// ── Create the Axios instance ──────────────────────────────────────
// baseURL: every request we make will be prefixed with this.
// e.g.  api.post('/api/v1/auth/send-otp', ...)
//       → actually calls http://localhost:8000/api/v1/auth/send-otp
```

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || 'http://localhost:8000',
  headers: {
    'Content-Type': 'application/json',  // We always send/expect JSON
  },
  timeout: 15000,  // 15 seconds — give up if server takes too long
});


// ── Request interceptor — attach JWT automatically ──────────────────────
// This function runs on EVERY outgoing request.
api.interceptors.request.use(
  (config) => {
    // Read the token from localStorage (where we stored it after login).
    const token = localStorage.getItem('lexis_access_token');

    if (token) {
      // Add the Authorization header.
      // The backend's get_current_user dependency reads "Bearer <token>".
      config.headers['Authorization'] = `Bearer ${token}`;
    }

    return config;  // Always return the config — this tells Axios to proceed
  },
  (error) => {
    // If something went wrong while building the request, just reject it.
    return Promise.reject(error);
  }
);


// ── Response interceptor — handle 401 (expired / invalid token) ──────────
// This function runs on EVERY response that comes back from the server.
api.interceptors.response.use(
  (response) => {
    // If the response was successful (2xx), just pass it through unchanged.
    return response;
  },
  (error) => {
    // Check if the server returned 401 Unauthorized.
    if (error.response && error.response.status === 401) {
      // Clear stored credentials — they are no longer valid.
      localStorage.removeItem('lexis_access_token');
      localStorage.removeItem('lexis_refresh_token');
      localStorage.removeItem('lexis_user');

      // Send the user to the login page.
      // We use window.location.href (a hard browser redirect) instead of
      // React Router here because this interceptor lives outside React.
      if (window.location.pathname !== '/login') {
        window.location.href = '/login';
      }
    }

    // Re-throw the error so the calling code can handle it too if needed.
    return Promise.reject(error);
  }
);


// ── Typed API helpers ─────────────────────────────────────────────────────
// These thin wrappers call the exact backend endpoints from Module 2.
// Keeping them here means if the URL ever changes, you only fix it in one place.

/**
 * Step 1 of login: request an OTP for an email address (or phone number).
 * @param {string|object} target  Email address (e.g. "citizen@example.com") or { email } / { ph
    one }
 * @returns {Promise}             Resolves with the axios response object
 */
export function sendOtp(target) {
  const payload = typeof target === 'string'
    ? (target.includes('@') ? { email: target.trim().toLowerCase() } : { phone: target.trim() })
```

```
    : target;
  return api.post('/api/v1/auth/send-otp', payload);
}

/**
 * Step 2 of login: submit the OTP received.
 * @param {string|object} target  Email address or phone number, or object { email, otp }
 * @param {string} [otp]          6-digit code
 * @returns {Promise}             Resolves with { access_token, refresh_token, role }
 */
export function verifyOtp(target, otp) {
  let payload;
  if (typeof target === 'object') {
    payload = target;
  } else if (typeof target === 'string' && target.includes('@')) {
    payload = { email: target.trim().toLowerCase(), otp: otp?.trim() };
  } else {
    payload = { phone: target?.trim(), otp: otp?.trim() };
  }
  return api.post('/api/v1/auth/verify-otp', payload);
}

/**
 * Exchange Firebase ID token for LEXIS JWT access + refresh tokens (legacy).
 * @param {string} idToken  Firebase user ID token
 * @param {string} phone    Phone number verified by Firebase
 * @returns {Promise}       Resolves with { access_token, refresh_token, role }
 */
export function verifyFirebaseToken(idToken, phone) {
  return api.post('/api/v1/auth/verify-firebase-token', { id_token: idToken, phone });
}


/**
 * Get the currently-logged-in user's profile.
 * The request interceptor above attaches the Bearer token automatically.
 * @returns {Promise}  Resolves with the user object from MongoDB
 */
export function getMe() {
  return api.get('/api/v1/auth/me');
}

/**
 * Sends a citizen query to the LEXIS Orchestrator Agent core.
 * @param {string} query  The legal question
 * @returns {Promise}     Resolves with QueryResponse { query, final_response, status, intents,
    scratchpad }
 */
export function sendQuery(query) {
  return api.post('/api/v1/query', { query });
}

/**
 * Fetch available document templates.
 */
export function getDraftTemplates() {
  return api.get('/api/v1/drafts/templates');
}

/**
 * Start a new drafting session.
 * @param {string} docType  'rti_application' | 'legal_notice' | 'consumer_complaint'
 */
export function startDraftSession(docType) {
  return api.post('/api/v1/drafts/session', { doc_type: docType });
}

/**
 * Submit answer to current question in draft session.
 * @param {string} sessionId
 * @param {string} answer
 */
export function submitDraftAnswer(sessionId, answer) {
```